



**UNIVERSITI SAINS MALAYSIA**

School of Electrical and Electronics Engineering  
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

**EEM 441**

Control & Instrumentation Laboratory

## **Experiment 5**

INTERFACING LCD TO ALTERA DE2 BOARD

Date : \_\_\_\_\_

Group No. : \_\_\_\_\_

Group Members : \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

# Experiment 5

## LCD Interfacing to Altera DE2 Board

### Objectives

---

1. To use Verilog code as a design entry method in Altera Quartus II Software.
2. To interface an LCD module to the Altera DE2 board.

### Equipment / Software Required

---

1. Altera Quartus II software
  - version 9/10/12 for Cyclone II
  - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board

### Introduction

---

In recent years, LCDs have largely replaced simple LED displays: they can show numbers, characters, and graphics, and are relatively easy to program. Table 5.1 lists the LCD pins relevant to programming.

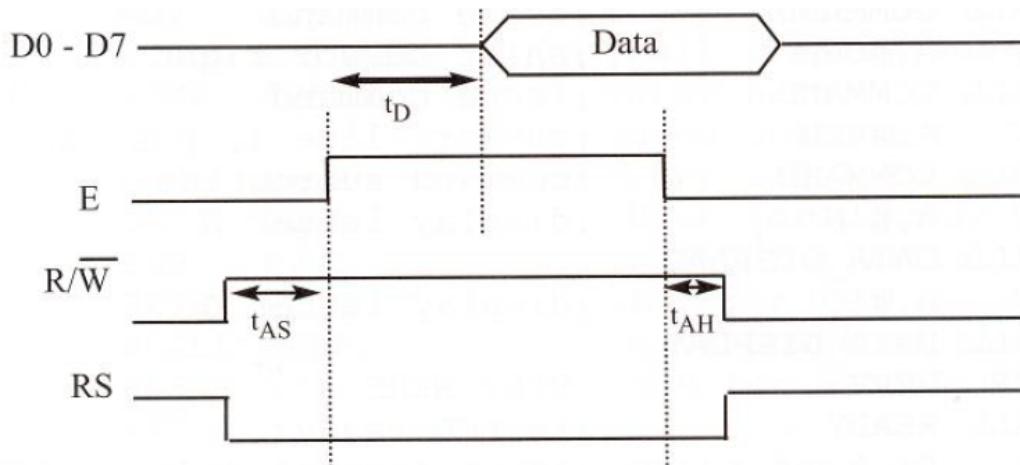
The *enable* (E) pin is used by the LCD to latch the information presented on its data pins: a high-to-low transition on E must be applied for the LCD to latch the data, and this pulse must be at least 450 ns wide. The required timing is shown in Figure 5.1.

Two important internal registers are selected via the RS pin. When  $RS = 0$ , the *instruction/command* register is selected, allowing commands to be sent to the LCD (Table 5.2 lists the available command codes). When  $RS = 1$ , the *data* register is selected, allowing ASCII character data to be written for display, sent through data pins D0–D7.

It is good practice to check the busy flag before writing to the LCD: with  $RS = 0$  and  $R/\overline{W} = 1$ , the busy flag appears on D7. While  $D7 = 1$  the LCD is busy with internal operations and will not accept new data; once  $D7 = 0$ , the LCD is ready to receive the next byte.

Table 5.1: Pin description for the LCD module.

| Pin | Symbol           | I/O | Description                                     |
|-----|------------------|-----|-------------------------------------------------|
| 1   | $V_{ss}$         | —   | Ground                                          |
| 2   | $V_{cc}$         | —   | +5 V power supply                               |
| 3   | $V_{ee}$         | —   | Power supply to control contrast                |
| 4   | RS               | I   | RS = 0: command register; RS = 1: data register |
| 5   | $R/\overline{W}$ | I   | 0: write; 1: read                               |
| 6   | E                | I/O | Enable (latches data on high-to-low transition) |
| 7   | DB0              | I/O | 8-bit data bus                                  |
| 8   | DB1              | I/O |                                                 |
| 9   | DB2              | I/O |                                                 |
| 10  | DB3              | I/O |                                                 |
| 11  | DB4              | I/O |                                                 |
| 12  | DB5              | I/O |                                                 |
| 13  | DB6              | I/O |                                                 |
| 14  | DB7              | I/O |                                                 |



$t_D$  = Data output delay time

$t_{AS}$  = Setup time prior to E (going high) for both RS and  $R/\overline{W}$  = 140 ns (minimum)

$t_{AH}$  = Hold time after E has come down for both RS and  $R/\overline{W}$  = 10 ns (minimum)

Note: Read requires an L-to-H pulse for the E pin.

Figure 5.1: LCD write-cycle timing diagram.

Table 5.2: LCD command codes.

| Code (hex) | Command to LCD instruction register   |
|------------|---------------------------------------|
| 1          | Clear display screen                  |
| 2          | Return home                           |
| 4          | Decrement cursor (shift cursor left)  |
| 6          | Increment cursor (shift cursor right) |
| 5          | Shift display right                   |
| 7          | Shift display left                    |
| 8          | Display off, cursor off               |
| A          | Display off, cursor on                |
| C          | Display on, cursor off                |
| E          | Display on, cursor blinking           |
| F          | Display on, cursor blinking           |
| 10         | Shift cursor position left            |
| 14         | Shift cursor position right           |
| 18         | Shift entire display left             |
| 1C         | Shift entire display right            |
| 80         | Force cursor to beginning of line 1   |
| C0         | Force cursor to beginning of line 2   |
| 38         | 2 lines, 5 × 7 dot matrix             |

## Procedure

### Activity 1 – Displaying lookup-table data on the LCD

*Objective: display fixed text, stored in a lookup table, on the LCD.*

1. Open Quartus II (Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N)).
2. Write the Verilog modules given in Listing 5.1 (lcd\_t) and Listing 5.2 (LCD\_Controller) to send commands and data to the LCD.
3. lcd\_t uses a lookup table (LUT) to store the sequence of commands and characters to send to the LCD.
4. Create a reusable symbol for each module, as in Experiment 3.
5. Draw the schematic in Figure 5.2, connecting lcd\_t's outputs to LCD\_Controller's inputs, and LCD\_Controller's outputs to the LCD interface pins.

6. Assign pin numbers as given in the Altera DE2 board manual (Cyclone II or Cyclone IV).
7. Download the program to the Altera DE2 board.
8. Discuss the result based on your observation of the LCD.

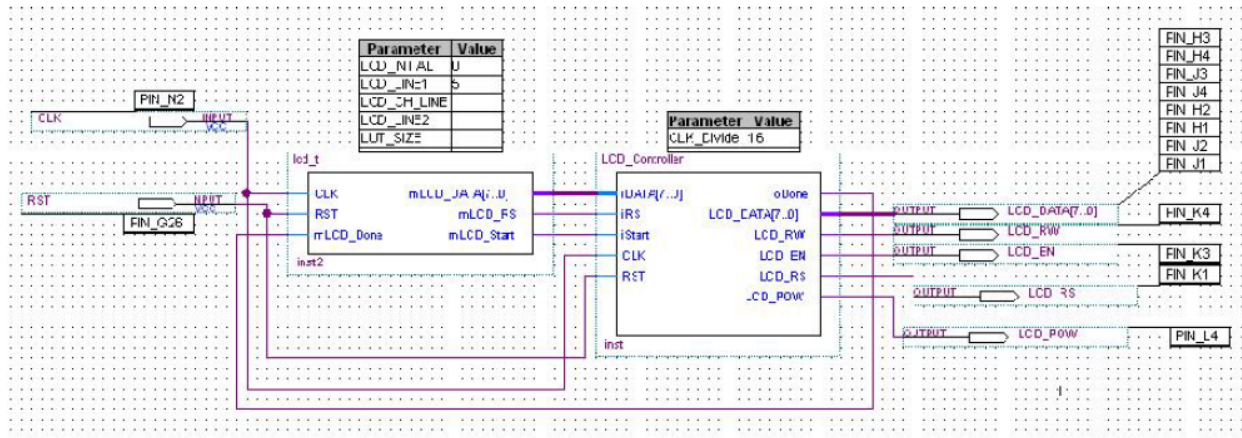


Figure 5.2: Schematic diagram for the LCD interface: `lcd_t` (lookup-table sequencer) feeding `LCD_Controller` (bus-timing controller).

**Table 5.3:** Pin assignment for the LCD interface.

| #  | Node name   | Direction | Location |
|----|-------------|-----------|----------|
| 1  | LCD_DATA[7] | Output    |          |
| 2  | LCD_DATA[6] | Output    |          |
| 3  | LCD_DATA[5] | Output    |          |
| 4  | LCD_DATA[4] | Output    |          |
| 5  | LCD_DATA[3] | Output    |          |
| 6  | LCD_DATA[2] | Output    |          |
| 7  | LCD_DATA[1] | Output    |          |
| 8  | LCD_DATA[0] | Output    |          |
| 9  | LCD_EN      | Output    |          |
| 10 | LCD_POW     | Output    |          |
| 11 | LCD_RS      | Output    |          |
| 12 | LCD_RW      | Output    |          |
| 13 | CLK         | Input     |          |
| 14 | RST         | Input     |          |

## lcd\_t.v – Appendix 1: lookup-table sequencer (part 1 of 2)

```

1  module lcd_t (
2      // Host side
3      CLK, RST, mLCD_Done,
4      mLCD_DATA, mLCD_RS, mLCD_Start
5  );
6      // Host side
7      input  CLK, RST, mLCD_Done;
8      // Internal wires/registers
9      output [7:0] mLCD_DATA;
10     output mLCD_RS, mLCD_Start;
11     reg  [5:0] LUT_INDEX;
12     reg  [8:0] LUT_DATA;
13     reg  [5:0] mLCD_ST;
14     reg  [17:0] mDLY;
15     reg  mLCD_Start;
16     reg  [7:0] mLCD_DATA;
17     reg  mLCD_RS;
18     wire mLCD_Done;
19
20     parameter LCD_INITIAL = 0;
21     parameter LCD_LINE1   = 5;
22     parameter LCD_CH_LINE = LCD_LINE1 + 16;
23     parameter LCD_LINE2   = LCD_LINE1 + 16 + 1;
24     parameter LUT_SIZE    = LCD_LINE1 + 32 + 1;
25
26     // Sequencing state machine: send one LUT entry, wait for
27     // LCD_Controller to finish (mLCD_Done), pause, then advance.
28     always @(posedge CLK or negedge RST)
29     begin
30         if (!RST)
31         begin
32             LUT_INDEX  <= 0;
33             mLCD_ST     <= 0;
34             mDLY        <= 0;
35             mLCD_Start <= 0;
36             mLCD_DATA  <= 0;
37             mLCD_RS     <= 0;
38         end
39         else if (LUT_INDEX < LUT_SIZE)
40         begin
41             case (mLCD_ST)
42             0: begin
43                 mLCD_DATA <= LUT_DATA[7:0];
44                 mLCD_RS   <= LUT_DATA[8];
45                 mLCD_Start <= 1;
46                 mLCD_ST    <= 1;
47             end
48             1: if (mLCD_Done) begin
49                 mLCD_Start <= 0;
50                 mLCD_ST    <= 2;

```

```

51         end
52

```

#### lcd\_t.v — Appendix 1: lookup-table sequencer (part 2 of 2, continued)

```

1      2: if (mDLY < 18'h3FFFE)
2          mDLY <= mDLY + 1;
3      else begin
4          mDLY <= 0;
5          mLCD_ST <= 3;
6      end
7      3: begin
8          LUT_INDEX <= LUT_INDEX + 1;
9          mLCD_ST <= 0;
10     end
11     endcase
12     end
13     end
14     // Lookup table: LCD_INITIAL entries are setup commands
15     // (function set, display on, clear, entry mode, home);
16     // LCD_LINE1 / LCD_LINE2 entries are ASCII characters for
17     // each line, with the MSB (bit 8) marking RS = 1 (data).
18     always
19     begin
20         case (LUT_INDEX)
21             LCD_INITIAL+0: LUT_DATA <= 9'h038; // function set
22             LCD_INITIAL+1: LUT_DATA <= 9'h00C; // display on
23             LCD_INITIAL+2: LUT_DATA <= 9'h001; // clear display
24             LCD_INITIAL+3: LUT_DATA <= 9'h006; // entry mode set
25             LCD_INITIAL+4: LUT_DATA <= 9'h080; // cursor home, line 1
26             // Line 1: "Welcome to the "
27             LCD_LINE1+0: LUT_DATA <= 9'h120;
28             LCD_LINE1+1: LUT_DATA <= 9'h157;
29             LCD_LINE1+2: LUT_DATA <= 9'h165;
30             // ... remaining characters follow the same pattern ...
31             LCD_LINE1+15: LUT_DATA <= 9'h120;
32             LCD_CH_LINE: LUT_DATA <= 9'h0C0; // move to line 2
33             // Line 2: "    EEM 441 Lab "
34             LCD_LINE2+0: LUT_DATA <= 9'h120;
35             LCD_LINE2+4: LUT_DATA <= 9'h145;
36             // ... remaining characters follow the same pattern ...
37             LCD_LINE2+15: LUT_DATA <= 9'h120;
38             default: LUT_DATA <= 9'h000;
39         endcase
40     end
41 endmodule
42

```

Listing 5.1: lcd\_t.v — lookup-table LCD message sequencer.

## LCD\_Controller.v – Appendix 2: bus-timing controller (part 1 of 2)

```

1  module LCD_Controller (
2      // Host side
3      iDATA, iRS, iStart, oDone, CLK, RST,
4      // LCD interface
5      LCD_DATA, LCD_RW, LCD_EN, LCD_RS, LCD_POW
6  );
7      parameter CLK_Divide = 16;
8
9      // Host side
10     input  [7:0] iDATA;
11     input  iRS, iStart;
12     input  CLK, RST;
13     output reg oDone;
14
15     // LCD interface
16     output [7:0] LCD_DATA;
17     output reg LCD_EN;
18     output LCD_RW;
19     output LCD_RS;
20     output LCD_POW;
21
22     // Internal registers
23     reg [4:0] Cont;
24     reg [1:0] ST;
25     reg preStart, mStart;
26
27     // Write-only interface: bypass iRS straight to LCD_RS
28     assign LCD_DATA = iDATA;
29     assign LCD_RW    = 1'b0;
30     assign LCD_RS    = iRS;
31     assign LCD_POW   = 1'b1;
32     always @(posedge CLK or negedge RST)
33     begin
34         if (!RST)
35         begin
36             oDone    <= 1'b0;
37             LCD_EN    <= 1'b0;
38             preStart  <= 1'b0;
39             mStart    <= 1'b0;
40             Cont      <= 0;
41             ST        <= 0;
42         end
43         else
44         begin
45             // Rising-edge detect on iStart
46             preStart <= iStart;
47             if ({preStart, iStart} == 2'b01)
48             begin
49                 mStart <= 1'b1;
50                 oDone  <= 1'b0;

```



```
51     end
```

#### LCD\_Controller.v – Appendix 2: bus-timing controller (part 2 of 2, continued)

```

1      if (mStart)
2      begin
3          case (ST)
4              0: ST <= 1;                // wait setup
5              1: begin
6                  LCD_EN <= 1'b1;        // raise enable
7                  ST <= 2;
8              end
9              2: if (Cont < CLK_Divide)  // hold enable pulse width
10                 Cont <= Cont + 1;
11                 else
12                     ST <= 3;
13              3: begin
14                  LCD_EN <= 1'b0;        // drop enable: latches data
15                  mStart <= 1'b0;
16                  oDone <= 1'b1;
17                  Cont <= 0;
18                  ST <= 0;
19              end
20          endcase
21      end
22  end
23  end
24  endmodule
```

Listing 5.2: LCD\_Controller.v – generates the LCD enable-pulse timing.

### Activity 2 – Displaying DIP switch input on the LCD

*Objective: display the live 8-bit DIP switch configuration on the LCD.*

1. Write Verilog code that reads the 8-bit DIP switch configuration and displays it on the LCD (e.g. if switch 1 is turned on, the LCD should display 1 in the corresponding position).
2. Print and submit the Verilog code and simulation result.